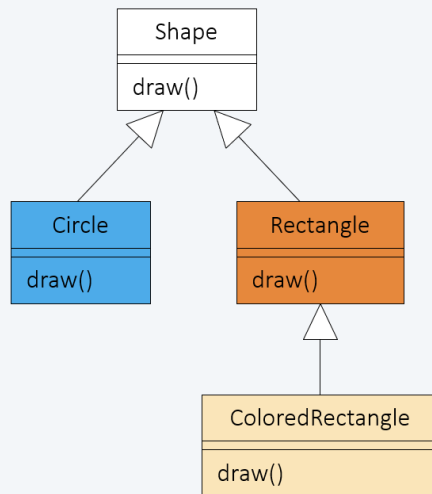


## 3 Objektorientierte Programmierung



3.1 Grundlegendes zur OOP

3.2 Vererbung

3.3 Polymorphismus

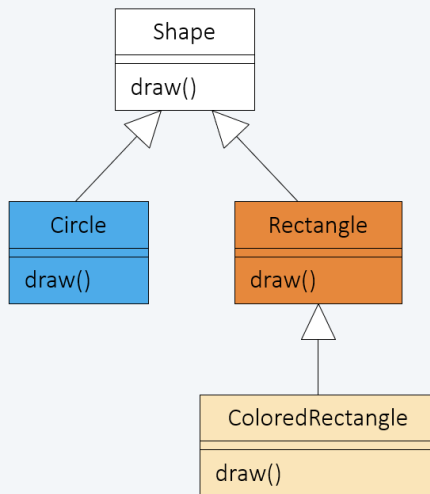
3.4 Dynamische Bindung

3.5 Abstrakte Klassen

3.6 Typtest und Typumwandlung

3.7 Vererbungsarten

## 3 Objektorientierte Programmierung



### 3.1 Grundlegendes zur OOP

### 3.2 Vererbung

### 3.3 Polymorphismus

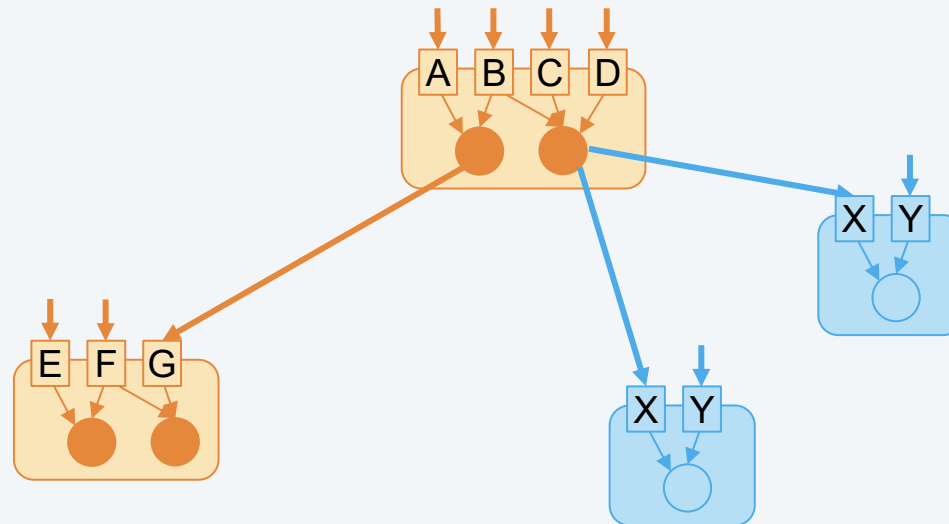
### 3.4 Dynamische Bindung

### 3.5 Abstrakte Klassen

### 3.6 Typtest und Typumwandlung

### 3.7 Vererbungsarten

- im Mittelpunkt steht die Auffassung, dass **Daten und Operationen Einheiten**, nämlich Objekte, bilden
- ein Programmsystem kann als Sammlung von miteinander **kommunizierenden Objekten** angesehen werden
- zur Laufzeit besteht ein solches Programmsystem aus einem **Netzwerk von Objekten**, die gemeinsam an der Lösung des Problems arbeiten



## Verminderung der **semantischen Lücke**

- durchgängiger methodischer Ansatz verkleinert die semantische Lücke zwischen Anwenderwelt und Implementierung
- Identifikation und Modellierung problemadäquater Objekte in allen Entwicklungsphasen

## Verbesserung der **Wiederverwendbarkeit**

- Realisierung von Komponenten mit flexiblen Halbfertigfabrikaten
- Konstruktion neuer Systeme durch Kombination bereits existierender Komponenten und Halbfertigfabrikaten

## Verbesserung der **Änderbarkeit und Erweiterbarkeit**

- inkrementelle Erweiterung und Anpassung von Systemkomponenten ohne Änderung ihres Quellcodes
- systemweit wirksame funktionale Anpassungen

- **Vererbung**. Klassen und damit die aus ihnen erzeugten Objekte können (z. B. durch Ableitung) die in anderen Klassen definierten Eigenschaften und Operationen erben und zu den geerbten Eigenschaften und Operationen können weitere hinzugefügt werden.
- **Polymorphie**. Variablen vom Typ einer Klasse können nicht nur Objekte dieser, sondern auch Objekte aller direkt oder indirekt davon abgeleiteten Klassen referenzieren. Damit können Variablen Objekte unterschiedlicher (polymorpher) Gestalt repräsentieren.
- **Dynamische Bindung**. Die Bindung von Methodenaufrufen mit einem bestimmten Codestück kann auch erst zur Laufzeit durchgeführt werden. Für gleiche Operationen können also verschiedene Algorithmen vorgesehen werden.

Darüber hinaus gehören zu OOP (aus Komfortgründen) auch noch weitere Konzepte.

## 1970er Jahre: Strukturierte Programmierung

- Prozeduren zur Strukturierung von Softwaresystemen
- Vermeidung unbeschränkter Ablaufstrukturen in Algorithmen
- Einführung von Datentypen
- basierend auf den Sprachentwicklungen Algol, Pascal

Ziel: Qualitätssteigerung  
und Meisterung der  
Komplexität

## 1980er Jahre: Modulatorientierte Programmierung

- Moduln zur Strukturierung von Softwaresystemen
- Abstrakte Datentypen, *information hiding*
- basierend auf den Sprachentwicklungen Ada, Modula-2

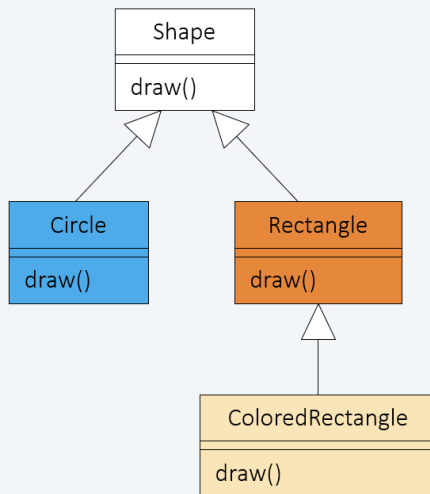
Ziel: Qualitätssteigerung  
und arbeitsteilige  
Softwareentwicklung

## 1990er Jahre: Objektorientierte Programmierung

- Klassen zur Strukturierung von Softwaresystemen
- erweiter- und veränderbare abstrakte Datentypen
- Möglichkeit der Faktorisierung von Gemeinsamkeiten durch Vererbungsmechanismus
- Steigerung der Flexibilität durch Polymorphie und dynamische Bindung
- basierend auf den Sprachentwicklungen Smalltalk, C++, Java

Ziel: Produktivitäts- und  
Qualitätssteigerung

## 3 Objektorientierte Programmierung



3.1 Grundlegendes zur OOP

3.2 Vererbung

3.3 Polymorphismus

3.4 Dynamische Bindung

3.5 Abstrakte Klassen

3.6 Typtest und Typumwandlung

3.7 Vererbungsarten

# Klassen (Wdh. aus Kapitel 2)

Sichtbarkeit

```
public class Color {
```

```
    private int red;  
    private int green;  
    private int blue;
```

Datenkomponenten

```
    public Color() {  
        this(0, 0, 0);  
    }
```

Konstruktoren

```
    public Color(int red, int green, int blue) {  
        this.red = red; this.green = green; this.blue = blue;  
    }
```

```
    public boolean equals(Color c) {  
        return red == c.red && green == c.green && blue == c.blue;  
    }
```

```
    public int getRed() { return red; }
```

```
    public void darker() { ... }
```

```
    ...
```

```
}
```

Methoden



Ein fundamentales und für OOP charakteristisches Konzept ist das **Vererbungskonzept**:

- Klassen (Typen) und damit die aus ihnen erzeugten Objekte können bereits **anderswo definierte Eigenschaften und Operationen erben** und zu diesen Eigenschaften und Operationen weitere hinzufügen
- Hypothese: Viele **Klassen sind einander ähnlich** und unterscheiden sich nur in Details
- Vererbung (*inheritance*) gestattet die Bildung neuer Klassen **ausgehend von bestehenden Klassen**

## Beispiele

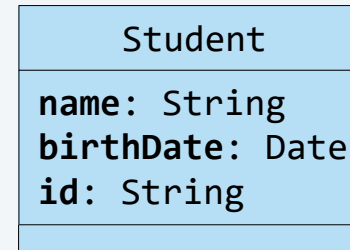
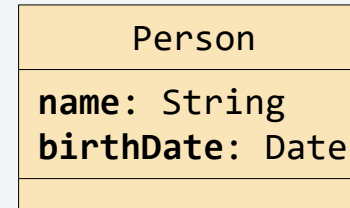
- ein Objekt das Studentendaten repräsentiert hat alle Eigenschaften eines Objekts das Personendaten repräsentiert **und ein Personenkennzeichen**
- ein farbiges Rechteck hat alle Eigenschaften eines gewöhnlichen Rechtecks **und zusätzlich eine Farbe**

## Beispiel **ohne Vererbung**

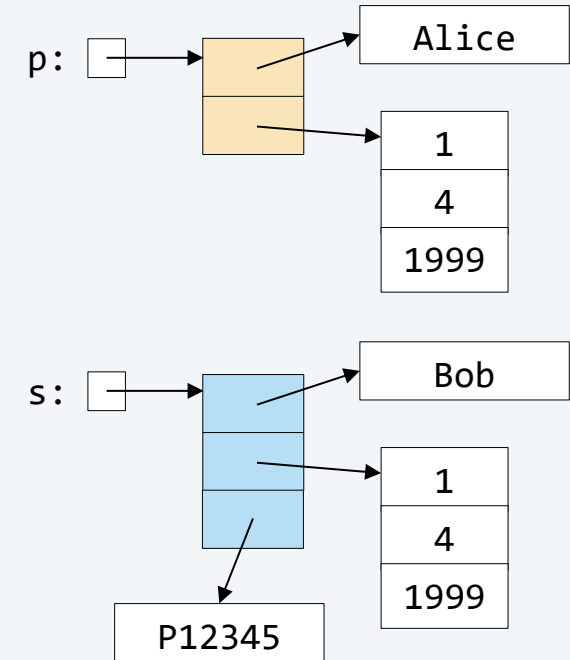
```
public class Person {  
    private String name;  
    private Date birthDate;  
    ...  
}
```

```
public class Student {  
    private String name;  
    private Date birthDate;  
    private String id;  
    ...  
}
```

## Klassen:



## Objekte:



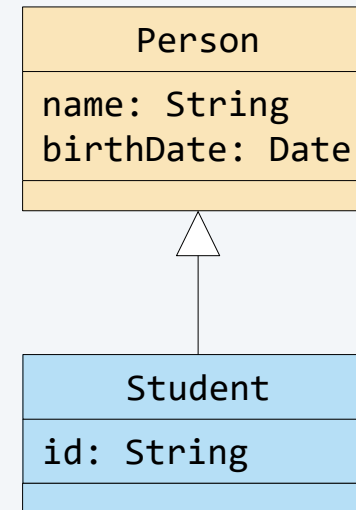
- Methoden der Klasse Person zur Manipulation der Datenkomponenten name und birthDate müssen auch in der Klasse Student implementiert werden (doppelter Code)
- eine Änderung der Klasse Person (z.B. neue Datenkomponenten, Operationen) zieht meist auch Änderung der Klasse Student mit sich

## Beispiel **mit Vererbung**

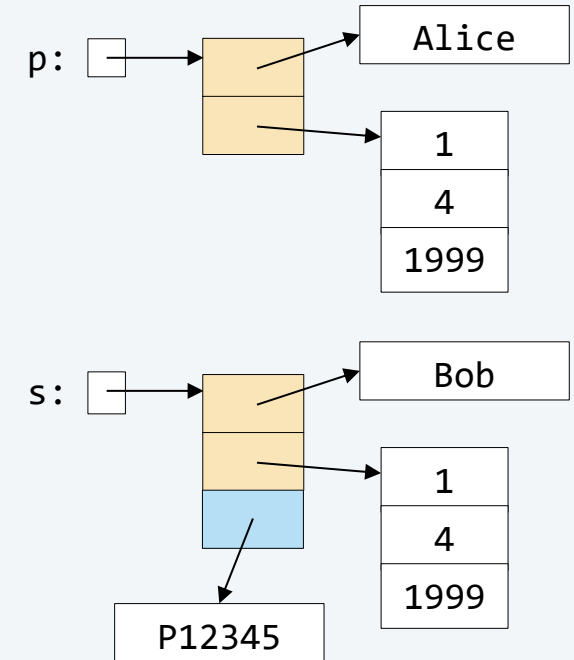
```
public class Person {  
    private String name;  
    private Date birthDate;  
    ...  
}
```

```
public class Student extends Person {  
    private String id;  
    ...  
}
```

## Klassen:



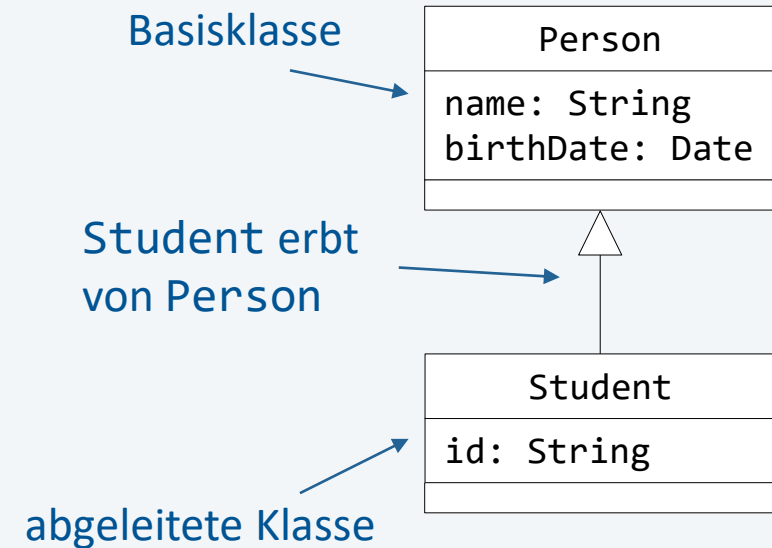
## Objekte:



- die Klasse Student erweitert die Klasse Person und „erbt“ alle Eigenschaften; in der Klasse Student wird eine neue Datenkomponente hinzugefügt
- ein Objekt der Klasse Student besitzt alle Eigenschaften der Klasse Person
- ein Objekt der Klasse Student **ist** auch ein Objekt der Klasse Person

## Klassen

- Person ... Basisklasse  
(*base class, super class*)
- Student ... abgeleitete Klasse  
(*derived class, sub class*)



## „ist-ein“ - Beziehung

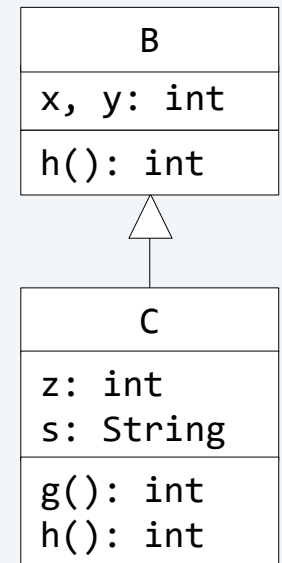
- durch Vererbung entsteht eine **ist-ein**-Beziehung zwischen Objekten: ein Objekt vom Typ B ist auch ein Objekt vom Typ A, wenn die Klasse B von der Klasse A (direkt od. indirekt) abgeleitet wurde
- anders ausgedrückt: ein Studentenobjekt kann alles, was ein Personenobjekt kann

Wenn von einer Basisklasse B eine neue Klasse C abgeleitet wird, erbt C alle Datenkomponenten und Operationen (Messages/Methoden), die B schon hat

Die abgeleitete Klasse C kann gegenüber ihrer Basisklasse B erweitert und adaptiert werden

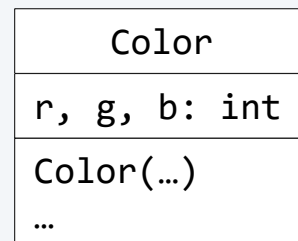
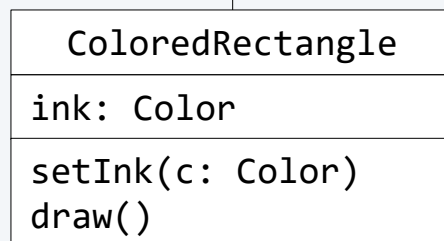
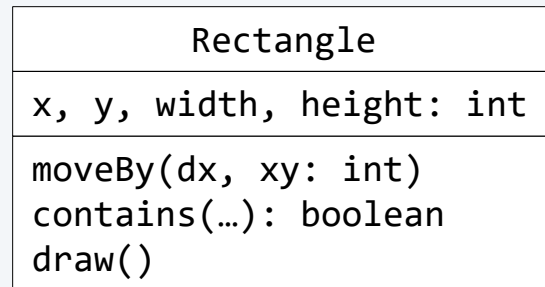
- neue Datenkomponenten (z, s) können **hinzugefügt** werden  
... damit kann das Eigenschaftsspektrum verbreitert werden
- neue Methoden (g) können **hinzugefügt** werden  
... damit kann das Funktionsspektrum verbreitert werden
- geerbte Methoden (h) können **überschrieben** (angepasst) werden  
... geerbte Funktionalität wird durch eine neue Methode ersetzt

Nicht möglich ist das Entfernen oder Ändern von Datenkomponenten oder Methoden!



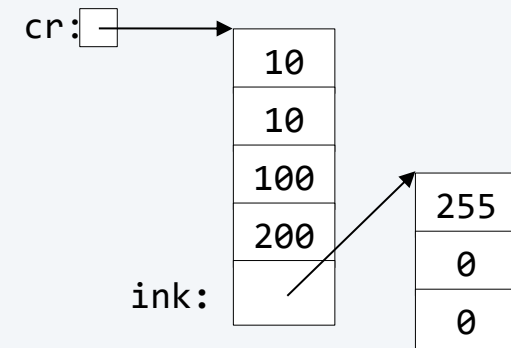
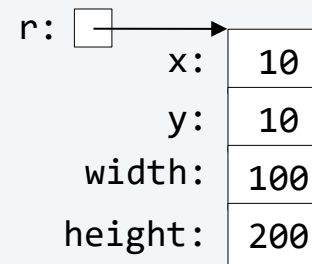
Beachte: Für die Message h an ein C-Objekt gibt es zwei Methoden!

- Bisherige Rectangle-Objekte wurden in einer Farbe (schwarz) gezeichnet
- Gesucht: eine Klasse ColoredRectangle für farbige Rechtecke
- Es ist naheliegend, die Klasse Rectangle als Basisklasse für die neue Klasse zu wählen



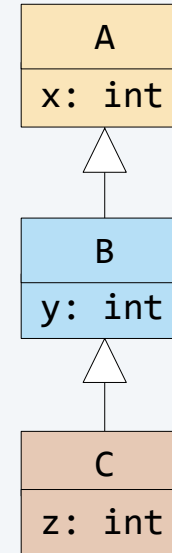
*ColoredRectangle* ist gegenüber *Rectangle* erweitert und adaptiert worden:

- eine neue Datenkomponente (ink) ist hinzugekommen
- eine neue Operation (setInk) ist zur Verfügung gestellt worden
- eine geerbte Operation (draw) ist modifiziert (überschrieben) worden



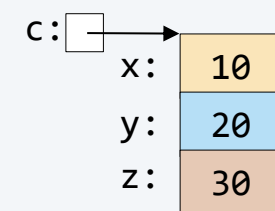
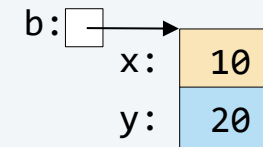
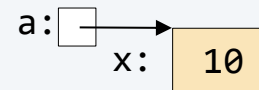
## Deklarationen

```
public class A {  
    private int x;  
}  
public class B extends A {  
    private int y;  
}  
public class C extends B {  
    private int z;  
}
```



## Verwendung

```
A a = new A();  
B b = new B();  
C c = new C();
```

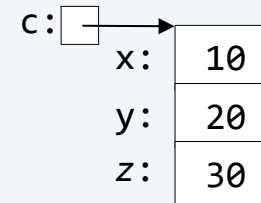
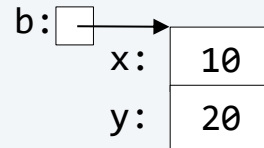
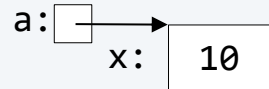


# Neue Methoden hinzufügen

## Verwendung

```
A a = new A();  
B b = new B();  
C c = new C();
```

## Objekte

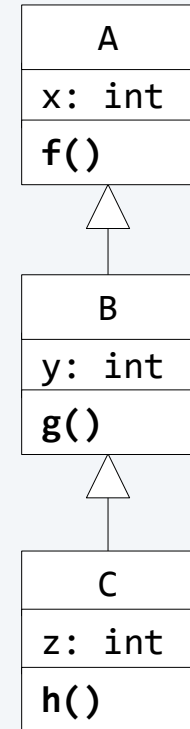


## Nachrichten

|       |   |
|-------|---|
| a.f() | ✓ |
| a.g() | ✗ |
| a.h() | ✗ |

|       |   |
|-------|---|
| b.f() | ✓ |
| b.g() | ✓ |
| b.h() | ✗ |

|       |   |
|-------|---|
| c.f() | ✓ |
| c.g() | ✓ |
| c.h() | ✓ |





## Klassen und Methoden



Methoden h berechnen Hash-Werte für B- oder C-Objekte

```
public class B {
    public int h() {
        return 31 * x + y;
    }
}
```

```
public class C extends B {
    public int h() {
        return 31*31*x + 31*y + z;
    }
}
```

## Objekte und Nachrichten

```
B b = new B(10, 20);
C c = new C(10, 20, 30);
```

| Message | Methoden |                                           |
|---------|----------|-------------------------------------------|
| b.h()   | B.h      | $31 \cdot 10 + 20$                        |
| c.h()   | C.h      | $31 \cdot 31 \cdot 10 + 31 \cdot 20 + 30$ |

## Klassen und Methoden



Methoden h berechnen Hash-Werte für B- oder C-Objekte

```
public class B {
    public int h() {
        return 31 * x + y;
    }
}
```

```
public class C extends B {
    public int h() {
        return 31 * super.h() + z;
    }
}
```

## Objekte und Nachrichten

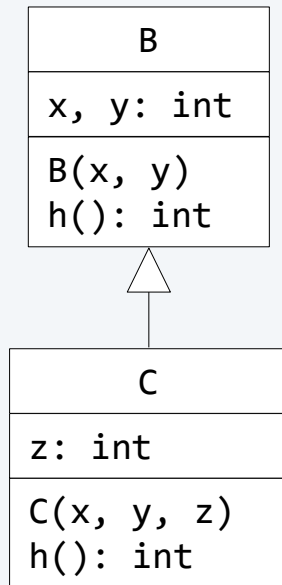
```
B b = new B(10, 20);
C c = new C(10, 20, 30);
```

| Message | Methoden |                      |
|---------|----------|----------------------|
| b.h()   | B.h      | 31*10 + 20           |
| c.h()   | C.h, B.h | 31*(31*10 + 20) + 30 |

# Konstruktoren(-aufruf) bei Vererbung

Konstruktoren der Basisklasse werden explizit aufgerufen

Klassen:

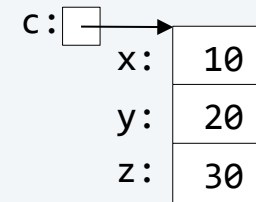
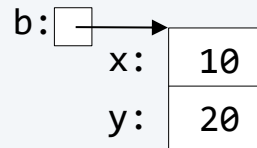


Konstruktoren:

```
public B(int x, int y) {
    this.x = x;
    this.y = y;
}
```

```
public C(int x, int y, int z) {
    super(x, y);
    this.z = z;
}
```

```
B b = new B(10, 20);
C c = new C(10, 20, 30);
```

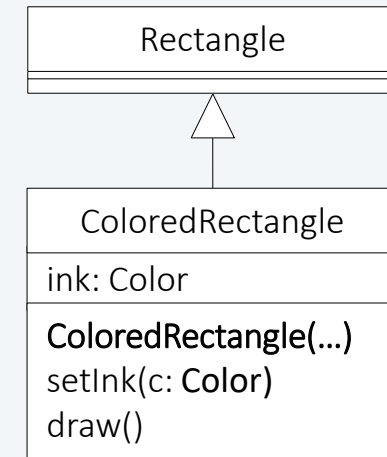


# Beispiel: Klasse ColoredRectangle

## Vervollständigung der Deklarationen

### Konstruktor

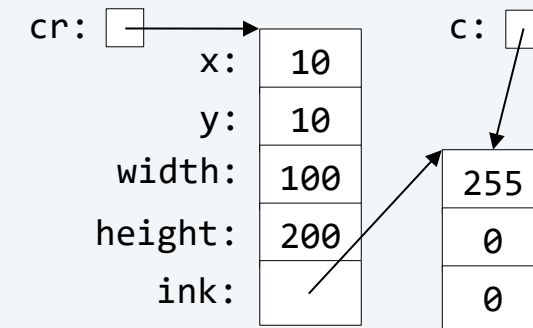
```
ColoredRectangle(int x, int y, ..., Color ink) {  
    super(x, y, width, height);  
    this.ink = ink;  
}
```



### Benutzung

```
Color c;  
ColoredRectangle cr;
```

```
c = new Color(255, 0, 0);  
cr = new ColoredRectangle(10, 10, 100, 200, c);
```



# Beispiel: Klasse ColoredRectangle

## Methode draw in Basisklasse Rectangle

```
public class Rectangle {  
    public void draw() {  
        StdDraw.rect(x, y, width, height);  
    }  
}
```

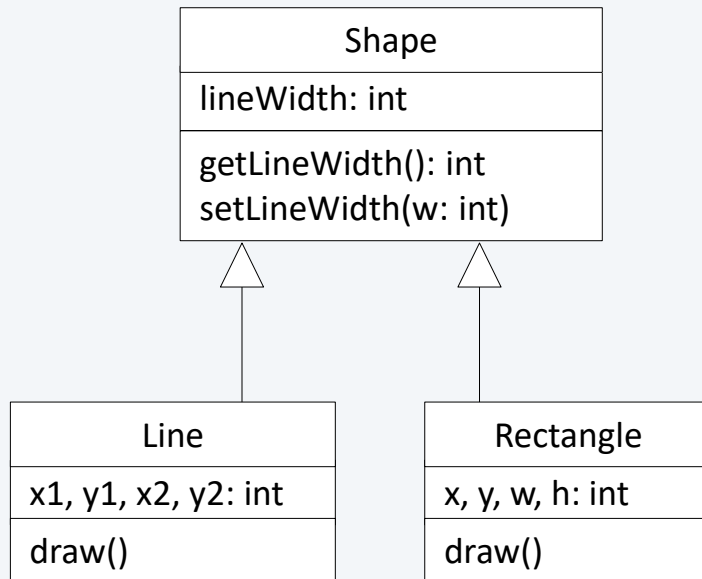


## Methode draw in abgeleiteter Klasse

```
public class ColoredRectangle extends Rectangle {  
    public void draw() {  
        if (ink != null) {  
            StdDraw.setColor(ink.getRed(), ink.getGreen(), ink.getBlue());  
        }  
        StdDraw.fillRect(getX(), getY(), getWidth(), getHeight());  
        super.draw();  
    }  
}
```



- allgemeine Eigenschaften (z.B. Linienstärke) in Basisklasse
- Zeichenoperationen in abgeleiteten Klassen (Shape kann nicht gezeichnet werden)



```
public class Shape {  
    public int getLineWidth() {  
        return lineWidth;  
    }  
}
```

```
public class Rectangle extends Shape {  
    public void draw() {  
        StdDraw.setPenWidth(getLineWidth());  
        StdDraw.rect(x, y, w, h);  
    }  
}
```

Die Klasse Object ist eine Oberklasse **aller** anderen Klassen

```
public class Color extends Object { ... }
```

```
public class Color { ... }
```

ohne extends ist die Basisklasse  
implizit Object

Alle Klassen- und Array-Typen **erben die Methoden** der Klasse Object

```
public class Object {  
    public Object() {}  
    public String toString() {  
        return getClass().getName() + "@" + Integer.toHexString(hashCode());  
    }  
    public boolean equals(Object obj) {  
        return (this == obj);  
    }  
    public int hashCode() {...}  
    public Class getClass() {...}  
    ...  
}
```

die Methoden **toString**, **equals**  
und **hashCode** sollten für alle  
Klassen überschrieben werden

```
Color c = new Color(250, 229, 185);
```

```
I0.println("c = " + c.toString());
```

c = Color@7344699f

## Überschreiben der Methode toString

```
public class Color extends Object {  
    private int red;  
    private int green;  
    private int blue;  
    public String toString() {  
        return "Color(r=%d,g=%d,b=%d)".formatted(red, green, blue);  
    }  
}
```

```
Color c = new Color(250, 229, 185);
```

```
I0.println("c = " + c.toString());
```

c = Color(250,229,185)

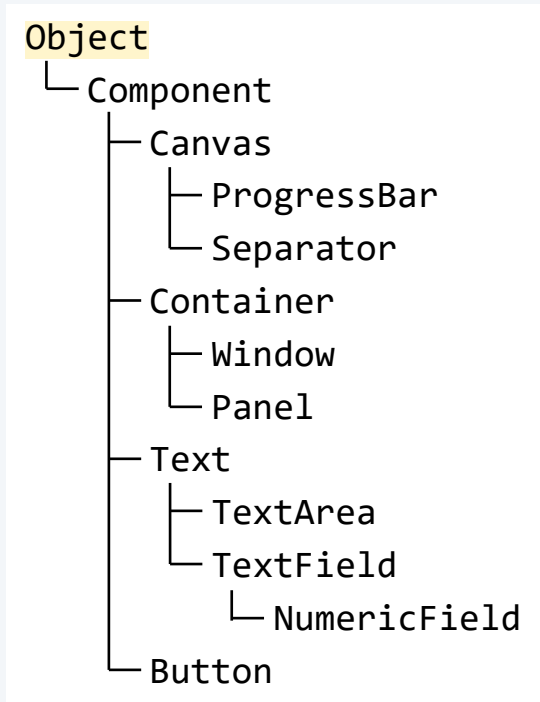
```
I0.println("c = " + c);
```

c = Color(250,229,185)

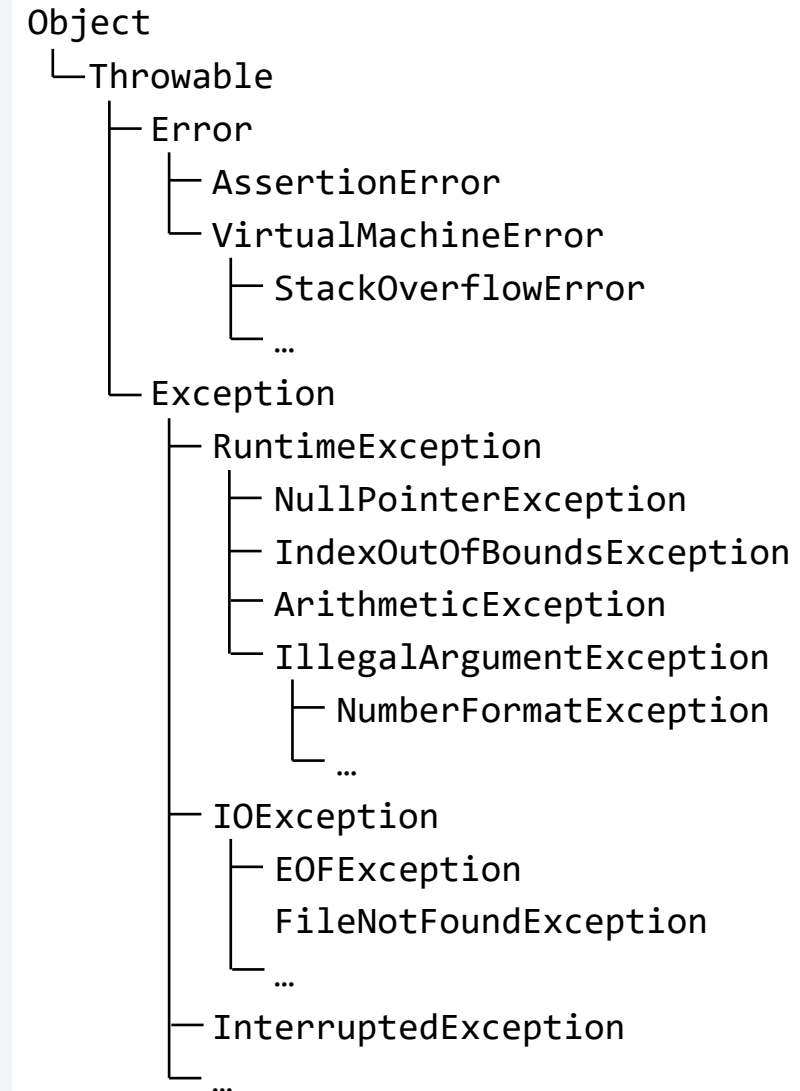
in diesem Kontext (Operator +) wird die  
Methode toString, automatisch aufgerufen!



## Einfache Vererbung führt zu Klassenhierarchien



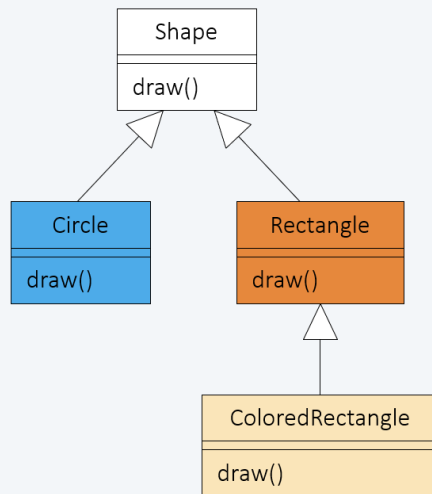
Abgeleitete Klassen sind im Allgemeinen spezieller als ihre Basisklassen



Vererbung wird benutzt um

- Codeverdopplungen bei ähnlichen abstrakten Datentypen zu vermeiden
- mit minimalem Aufwand neue Datentypen zu erzeugen, für die es schon Vorbilder gibt
- gemeinsame Schnittstellen für eine ganze Familie von Klassen zu definieren
- getesteten Code zu verwenden und zu erweitern, ohne ihn zu verändern

## 3 Objektorientierte Programmierung



3.1 Grundlegendes zur OOP

3.2 Vererbung

3.3 Polymorphismus

3.4 Dynamische Bindung

3.5 Abstrakte Klassen

3.6 Typtest und Typumwandlung

3.7 Vererbungsarten

- bisher (vor OOP) haben wir statisch typisierte Datenobjekte verwendet
- eine Variable a vom Typ A kann nur A-Objekte aufnehmen  
(= **monomorphe Variablen**)

```
A a;
```

```
int i;
```

```
String s;
```

- oft wären auch **polymorphe Variablen** wünschenswert und das lässt sich durch die bisher eingeführten Konzepte nun realisieren

## Polymorphe Variablen

- durch Vererbung von Klassen entsteht eine „**ist-ein**“-Beziehung zwischen Objekten
- eine Variable a vom Typ A kann deshalb Objekte der Klasse A **und aller davon abgeleiteten Klassen** aufnehmen

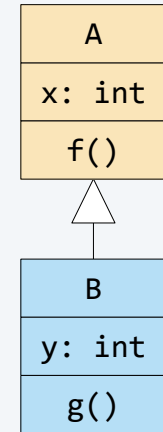
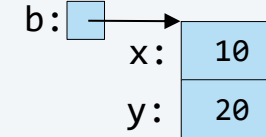
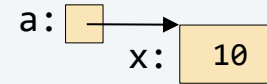
- in der objektorientierten Programmierung bedeutet Polymorphismus, dass mit Variablen vom Typ einer Basisklasse nicht nur Objekte dieser, sondern auch **Objekte aller direkt oder indirekt davon abgeleiteten Klassen** referenziert werden können
- damit können solche Variablen auf **Objekte unterschiedlicher Gestalt** verweisen, weshalb wir in dieser Situation von Polymorphismus sprechen
- diese Objekte haben allerdings eine Gemeinsamkeit: ihre Datentypen (Klassen) stammen **von derselben Basisklasse** ab
- der Datentyp einer Variablen kann sich somit zur Laufzeit ändern. Wir sprechen daher vom **dynamischen Datentyp**
- Variablen im OO Sinne besitzen also einen **statischen** und einen **dynamischen Datentyp**

# Beispiel: Polymorphe Variablen

## Bisher

```
A a;  
B b;
```

```
a = new A(10);  
b = new B(10, 20);
```



## Neu:

- `B` erbt alle Eigenschaften und Operationen von `A`
- Objekte der Klasse `B` „können“ also alles, was Objekte der Klasse `A` „können“ (eventuell auch mehr)

```
a = new B(30, 40);
```

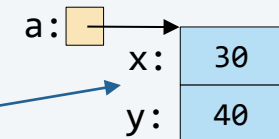
```
a.f()
```

✓

```
a.g()
```

✗

dynamischer Datentyp von `a` ist  
nun `B` (abgeleitet vom Typ `A`)

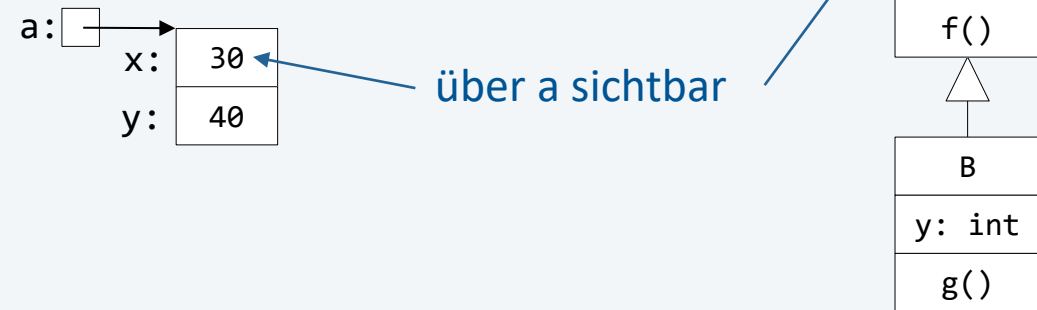


- über Variable `a` (vom stat. Typ `A`) kann nur auf Eigenschaften und Operationen des `B`-Objekts zugegriffen werden, die bereits in `A` definiert sind

## Statischer Typ (*static type*)

- wird in der Deklaration festgelegt und ist unveränderlich
- bestimmt, welche Elemente der Klasse verwendet werden können

```
A a;
```



## Dynamischer Typ (*dynamic type*)

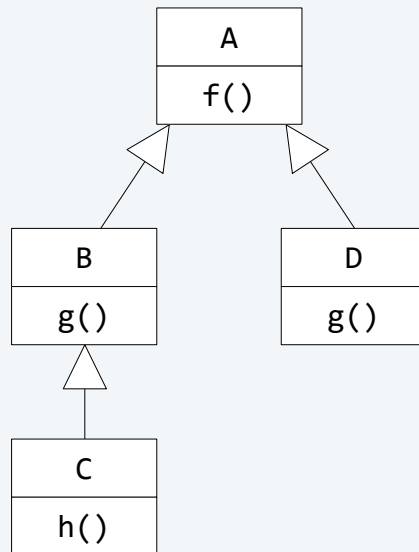
- ergibt sich durch die Zuweisung

```
a = new B(30 ↓40);
```

- kann sich durch eine neuerliche Zuweisung wieder ändern
- bestimmt, welche Methode aufgerufen wird (dynamische Bindung)

Objekte einer Klasse B dürfen einer Variable des Typs A zugewiesen werden, wenn B von A direkt oder indirekt abgeleitet wurde

Klassendiagramm:



Deklarationen:

```
A a; B b; C c; D d;
```

|                    |   |                                       |
|--------------------|---|---------------------------------------|
| <code>a = b</code> | ✓ | Variablen einer Wurzelklasse können   |
| <code>a = c</code> | ✓ | Objekte jeder abgeleiteten Klasse     |
| <code>a = d</code> | ✓ | aufnehmen                             |
| <code>b = a</code> | ✗ | B „kann mehr“ als A                   |
| <code>b = c</code> | ✓ |                                       |
| <code>b = d</code> | ✗ | D-Objekte sind keine B-Objekte        |
| <code>c = a</code> | ✗ | C „kann mehr“ als A und B             |
| <code>c = b</code> | ✗ |                                       |
| <code>c = d</code> | ✗ | C-Objekte sind keine D-Objekte        |
| <code>d = a</code> | ✗ | D „kann mehr“ als A                   |
| <code>d = b</code> | ✗ | B- und C-Objekte sind keine D-Objekte |
| <code>d = c</code> | ✗ |                                       |



```
Rectangle r;  
ColoredRectangle cr;
```

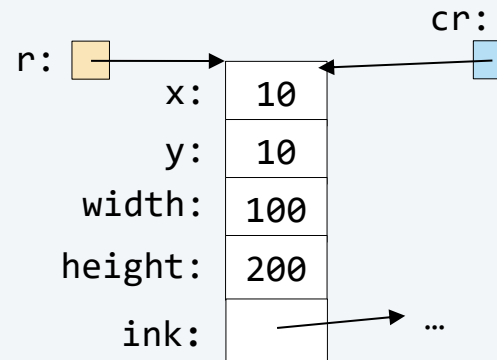
```
cr = new ColoredRectangle(...);  
r = cr;
```

```
r.draw();
```

✓

```
r.setInk();
```

✗

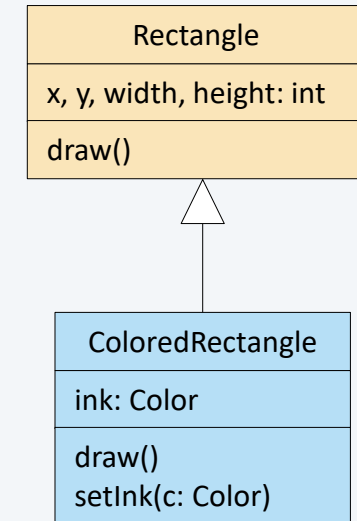


```
cr.draw();
```

✓

```
cr.setInk();
```

✓



- ein Objekt `r` (vom Typ `Rectangle`) kann nur die Nachricht `draw()` interpretieren
- ein Objekt `cr` (vom Typ `ColoredRectangle`) kann die Nachrichten `draw()` und `setInk()` interpretieren

- der dynamische Datentyp einer Variablen des statischen Datentyps X ist zur Übersetzungszeit nicht vorhersehbar

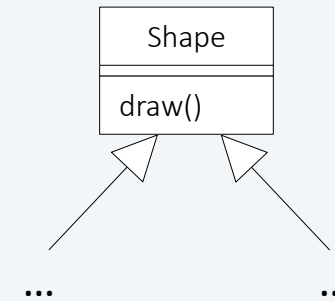
Shape item; ← welches Objekt verbirgt sich hinter item?

- Programmteile, die mit Objekten der Klasse X arbeiten, können ohne Änderung auch mit Objekten von abgeleiteten Klassen arbeiten

```
final static int MAX = ... ;
```

```
Shape[] shapes = new Shape[MAX];  
int n;
```

```
for (int i = 0; i < n ; i++) {  
    shapes[i].draw();  
}
```

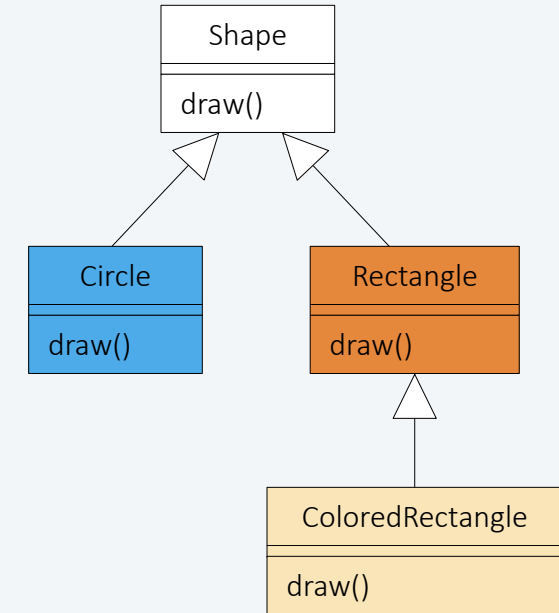
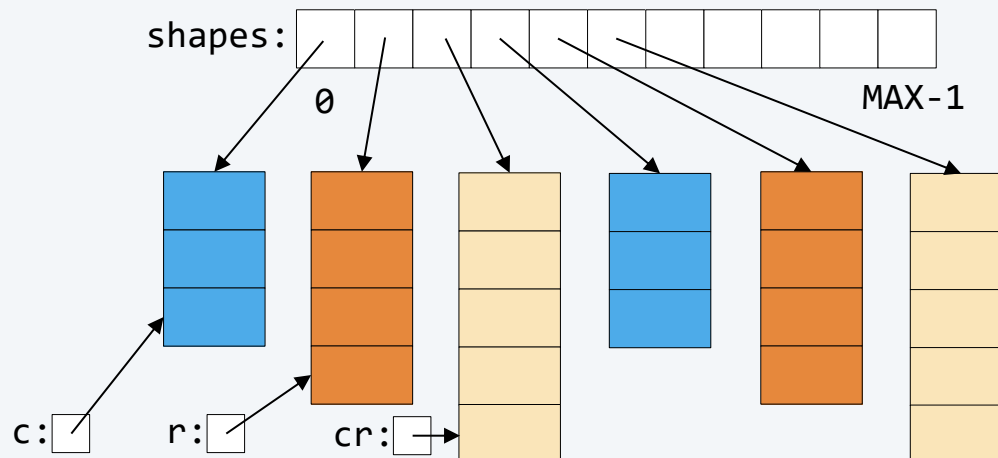


- die Hinzunahme neuer Shapes kann ohne Programmänderung erfolgen

## Beispiel: Sammlung geometrischer Objekte

```
Shape[] shapes = new Shape[MAX];  
Circle c; Rectangle r; ColoredRectangle cr;
```

```
c = new Circle(); shapes[0] = c;  
r = new Rectangle(); shapes[1] = r;  
cr = new ColoredRectangle(); shapes[2] = cr;  
shapes[3] = new Circle();  
shapes[4] = new Rectangle();  
shapes[5] = new ColoredRectangle();
```



Verwendung:

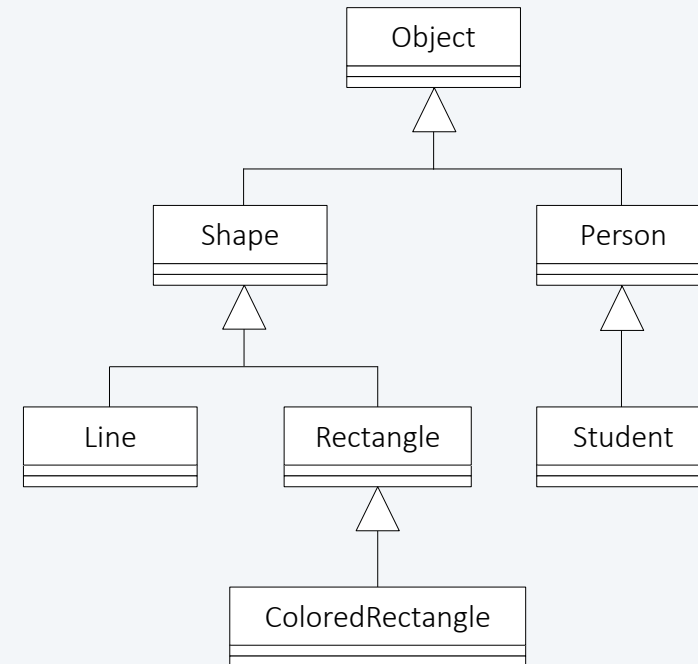
```
for (int i = 0; i < 6; i++) {  
    shapes[i].draw();  
}
```

Datenstrukturen (Listen, Bäume, ...) für Objekte beliebigen Typs

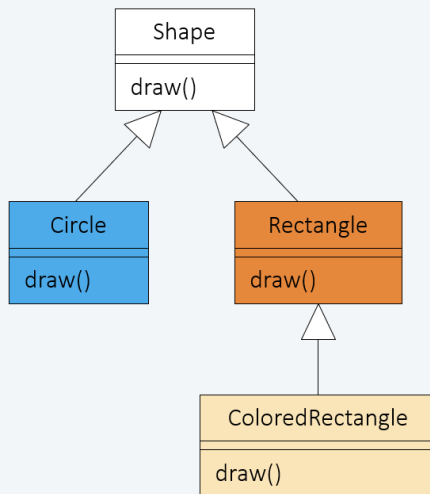
Alle Klassen/Typen müssen (direkt oder indirekt) von einer Wurzelklasse/Wurzeltyp abgeleitet werden

```
class ObjectStack {  
    private int top;  
    private Object[] data;  
    public      ObjectStack() {...}  
    public void  push(Object e) {...}  
    public Object pop()      {...}  
    public boolean isEmpty()  {...}  
}
```

```
ObjectStack s = new ObjectStack();  
s.push(new Line());  
s.push(new Person());  
s.push(new Student());
```



## 3 Objektorientierte Programmierung



3.1 Grundlegendes zur OOP

3.2 Vererbung

3.3 Polymorphismus

3.4 Dynamische Bindung

3.5 Abstrakte Klassen

3.6 Typtest und Typumwandlung

3.7 Vererbungsarten

Unter Bindung versteht man im Kontext von Programmiersprachen die **Verknüpfung des Aufrufs einer Operation** (einer Prozedur, Funktion oder Methode) mit der **auszuführenden Folge von Anweisungen**

In Abhängigkeit vom Zeitpunkt, wann die Bindung stattfindet, kann man zwei Ausprägungen unterscheiden:

- Bei nicht objektorientierten Programmiersprachen kann die Verknüpfung für Prozedur- und Funktionsaufrufe entweder **der Compiler** zur Übersetzungszeit (wenn das gesamte Programm in einer Übersetzungseinheit vorliegt) **oder der Binder (linker)** zur Bindezeit herstellen. In beiden Fällen erfolgt die Bindung vor der Programmausführung, weshalb diese Art von Bindung als **statische Bindung** bezeichnet wird.
- Bei objektorientierten Programmiersprachen gibt es (wegen des objektorientierten Paradigmas) auch Situationen, in denen die **Bindung** von Methodenaufrufen mit einem bestimmten Codestück **erst zur Laufzeit** durchgeführt werden kann, weshalb diese Art von Bindung als **dynamische Bindung** bezeichnet wird.

- Bindung = Verknüpfung zwischen Senden einer Nachricht und Auswahl einer Methode
- für monomorphe Variablen kann Methode zur Übersetzungszeit ausgewählt werden

```
A a;
```

```
a.f();
```

hinter a kann sich nur ein A-Objekt verbergen  
daher muss Methode A.f() ausgeführt werden

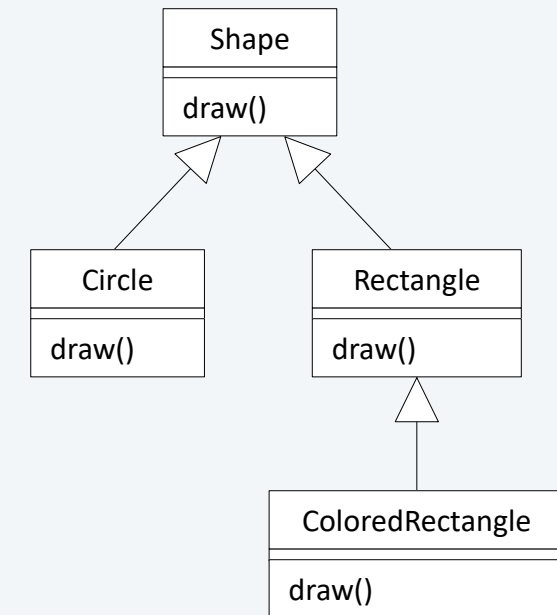
- bei polymorphen Variablen ist der dynamische Datentyp des Objekts zur Übersetzungszeit nicht bekannt
- die „richtige“ Methode kann erst zur Laufzeit ausgewählt werden
- **Auswahl** hängt vom **dynamischen Datentyp des Objekts** ab, das sich hinter einer Variablen verbirgt

- damit Nachricht `draw()` von Shape-Objekten „verstanden“ werden kann, muss `draw()` in Shape definiert werden

```
for (int i = 0; i < 6; i++) {  
    shapes[i].draw();  
}
```

- Dynamische Bindung

| Nachricht                     | Methode                              |
|-------------------------------|--------------------------------------|
| <code>shapes[0].draw()</code> | <code>Circle.draw()</code>           |
| <code>shapes[1].draw()</code> | <code>Rectangle.draw()</code>        |
| <code>shapes[2].draw()</code> | <code>ColoredRectangle.draw()</code> |
| <code>shapes[3].draw()</code> | <code>Circle.draw()</code>           |
| <code>shapes[4].draw()</code> | <code>Rectangle.draw()</code>        |
| <code>shapes[5].draw()</code> | <code>ColoredRectangle.draw()</code> |



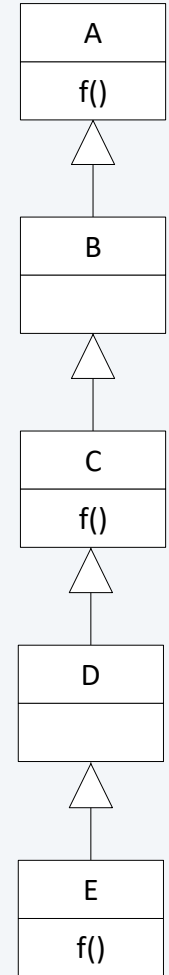
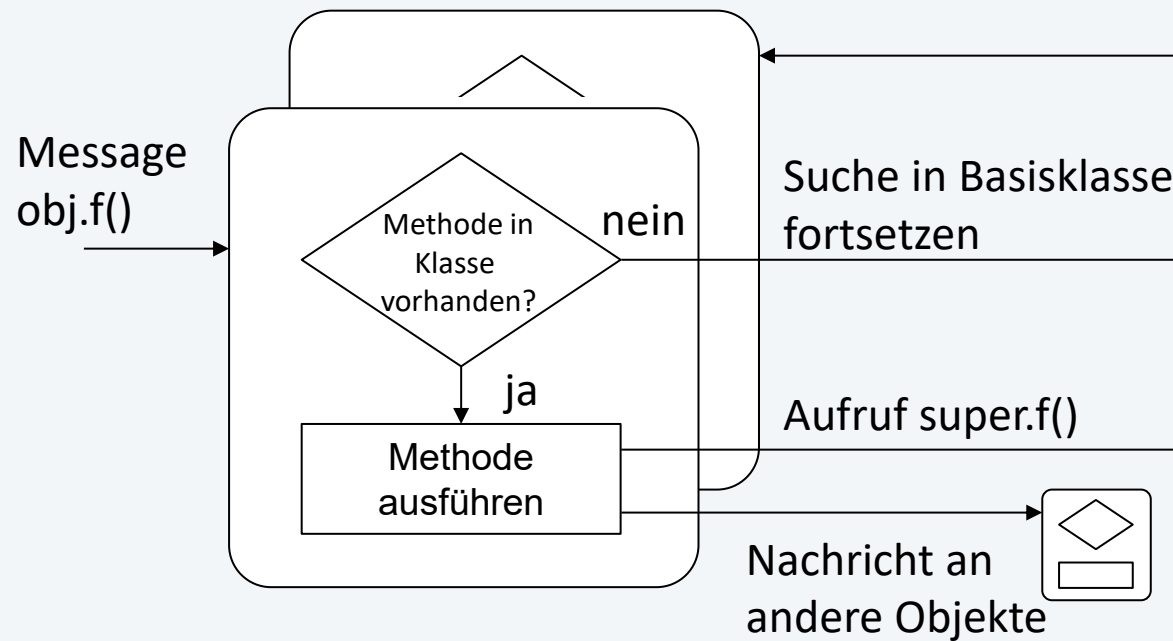
Beachte: `Shape.draw()` wird nie aufgerufen!



- bei konventioneller Programmierung sieht man im Quelltext, welcher Code ausgeführt wird
- das gilt bei objektorientierter Programmierung nicht mehr

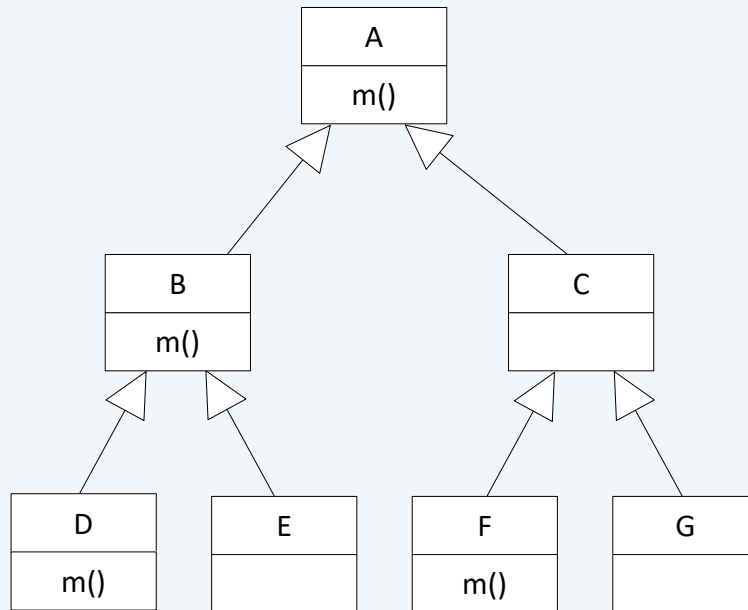
|             | <b>konventionell</b>                                  | <b>objektorientiert</b>                                           |
|-------------|-------------------------------------------------------|-------------------------------------------------------------------|
| Einheit     | Prozedur/Funktion                                     | Methode                                                           |
| Aktivierung | Aufruf<br><code>P()</code> oder <code>v := F()</code> | Nachricht<br><code>obj.p();</code> oder <code>v = obj.f();</code> |
| Bindung     | durch Namen                                           | durch Namen und Klasse des Empfängerobjektes                      |
| Zeitpunkt   | Übersetzung                                           | Laufzeit                                                          |
|             | Statische Bindung                                     | Dynamische Bindung                                                |

- Beginn bei Klasse des Empfänger-Objekts
- Ende bei direkter oder indirekter Basisklasse
- in typisierten Sprachen ist Methodensuche immer erfolgreich



**Beispiel:** Methode `m()` ist in den Klassen A, B, D und F implementiert

Klassendiagramm:



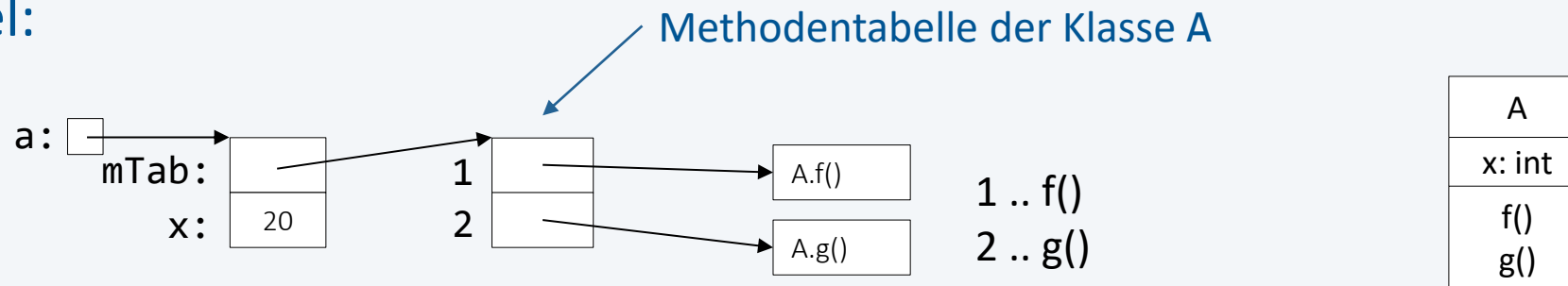
Methodensuche:

| Objekt  | Suchreihenfolge                 | Aufruf    |
|---------|---------------------------------|-----------|
| A,B,D,F | sofort in Klasse                | A.m(),... |
| C       | $C \rightarrow A$               | A.m()     |
| E       | $E \rightarrow B$               | B.m()     |
| G       | $G \rightarrow C \rightarrow A$ | A.m()     |

=> Methodensuche ist aufwändig

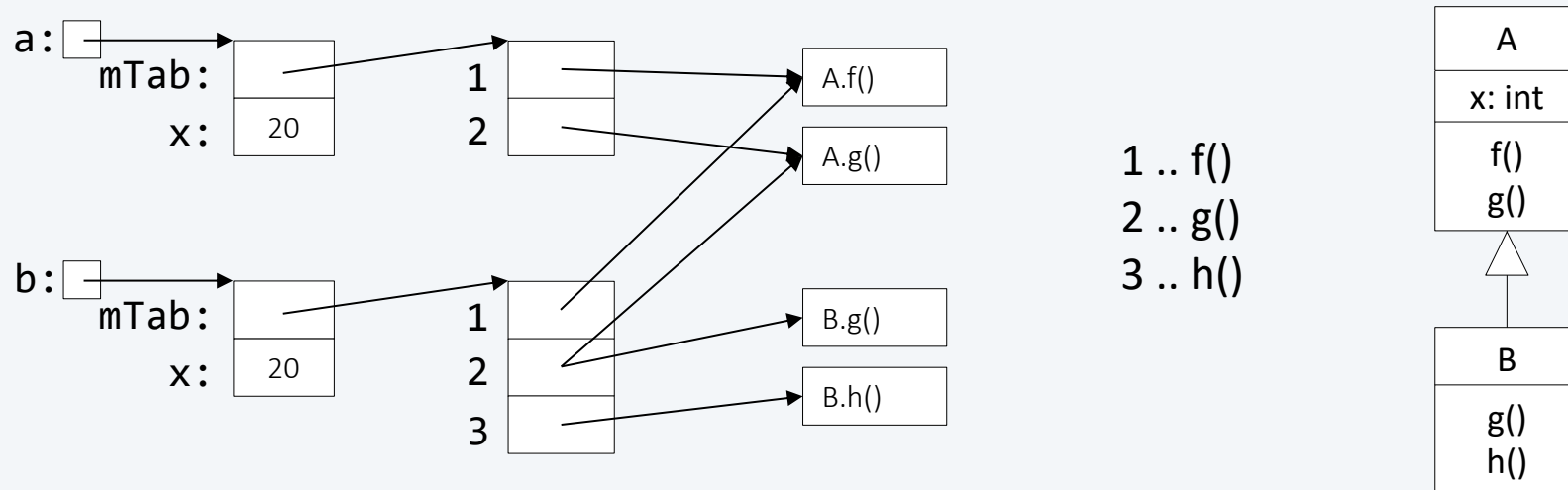
- Methoden werden nummeriert
- Adressen werden in **Methodentabelle** abgelegt
- jedes Objekt hat zusätzliche Datenkomponente mTab, die das Objekt mit der Methodentabelle verknüpft (mTab ist für Programmierer\*in nicht zugänglich)
- in allen Objekten ein und derselben Klasse zeigt mTab auf ein und dieselbe Methodentabelle
- für jede Klasse gibt es eine Methodentabelle

## Beispiel:



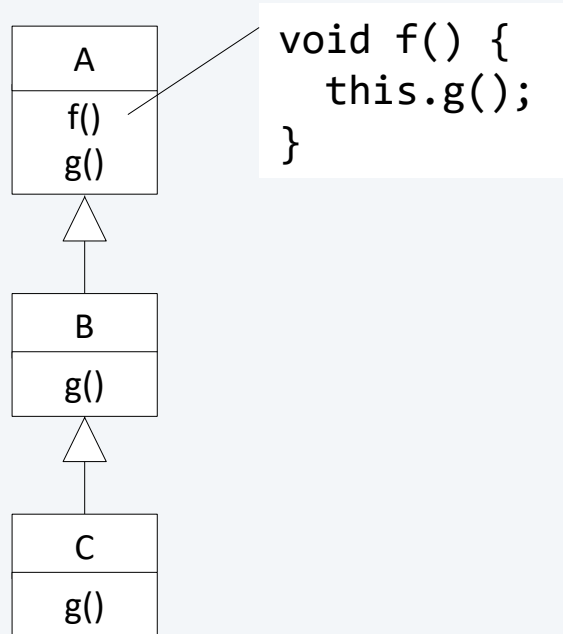
- für abgeleitete Klasse wird Methodentabelle der Basisklasse kopiert
- bei hinzugefügten Methoden wird die Methodentabelle erweitert
- Einträge für überschriebenen Methoden werden überschrieben
- Verwendung von Methodentabellen bedeutet weniger Aufwand bei Methodensuche

## Beispiel:



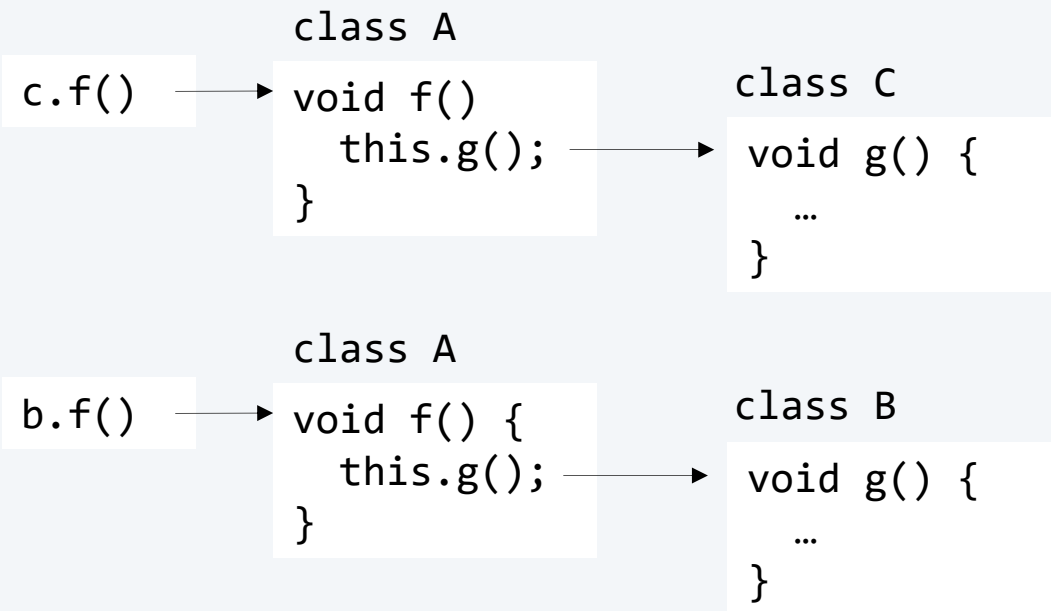
## Nachrichten an `this` werden dynamisch gebunden

Klassendiagramm:

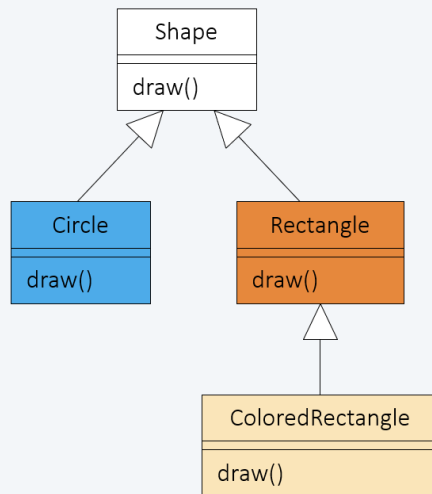


Aufrufe:

```
A b = new B();
A c = new C();
```



## 3 Objektorientierte Programmierung



3.1 Grundlegendes zur OOP

3.2 Vererbung

3.3 Polymorphismus

3.4 Dynamische Bindung

3.5 Abstrakte Klassen

3.6 Typtest und Typumwandlung

3.7 Vererbungsarten

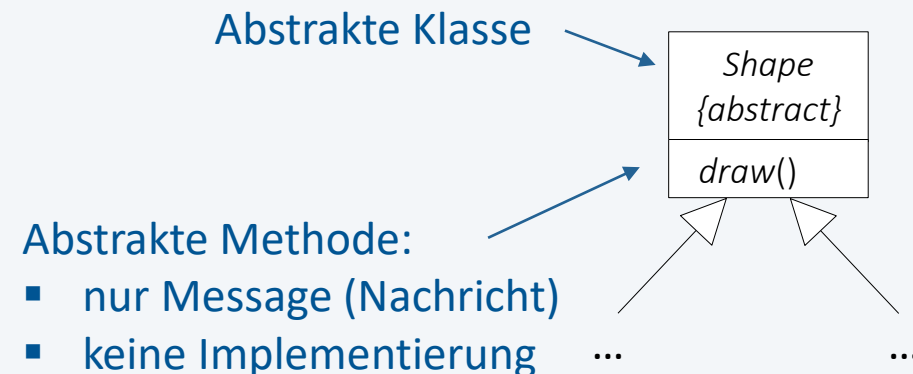
- zur Faktorisierung von Gemeinsamkeiten und Strukturierung ist es sinnvoll, abstrakte Klassen, d.h. Datentypen, zu denen es keine konkreten Objekte gibt, zu verwenden
- für Methoden solcher Klassen existiert u. U. keine sinnvolle Implementierung, Nachrichten sollen aber möglich sein

## Beispiel: Klasse Shape

- hinter `shapes[i]` verbirgt sich stets ein Objekt einer (konkreten) Klasse, die von der abstrakten Klasse Shape abgeleitet ist

```
Shape[] shapes = new Shape[MAX];  
int n;
```

```
for (int i = 0; i < n; i++) {  
    shapes[i].draw();  
}
```





## ■ Deklaration

```
public abstract class Shape {  
    public abstract void draw();  
}
```

eine Klasse ist abstrakt, wenn sie mit **abstract** gekennzeichnet ist

eine Methode ist abstrakt, wenn sie mit **abstract** gekennzeichnet ist und keine Implementierung hat

## ■ von abstrakten Klassen dürfen keine Objekte erzeugt werden

```
Shape s;           ✓  
s = new Shape();   ✗  
s.draw();          ✓
```

- die Methoden für die Messages abstrakter Klassen werden in den von ihnen abgeleiteten Klassen implementiert (oder abgeleitete Klassen sind selbst abstrakt)
- es können gemeinsame Schnittstellen für eine ganze Familie von Klassen realisiert werden

- Zusammenfassen von Gemeinsamkeiten verwandter Klassen
- Gemeinsamkeiten werden in (abstrakten) Basisklassen definiert

## Beispiel: Kreise, Rechtecke und Linien

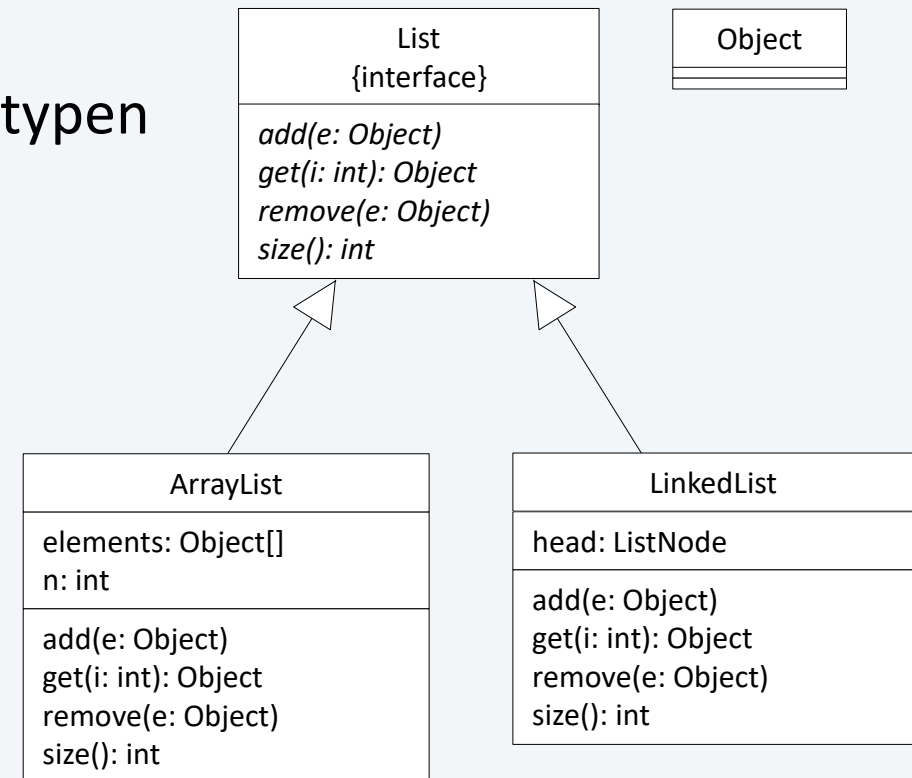
- haben ähnliche oder identische Eigenschaften u. Operationen, z.B.
  - Position und Ausdehnung
  - Zeichnen und Rotieren
  - Abfragen und Ändern von Position und Ausdehnung
- keine Klasse eignet sich als Basisklasse für eine andere
- Lösung:
  - Gemeinsamkeiten werden in abstrakter Basisklasse Shape definiert
  - abstrakte Messages für Operationen, die in Shape nicht (vollständig) implementiert werden können

- Faktorisierung kann zu Klassen führen, die ausschließlich abstrakte Operationen (Messages) definieren, d.h. ihre Schnittstellen haben
  - keine Methodenimplementierungen
  - keine Datenkomponenten

- OO Realisierung von abstrakten Datentypen

## Beispiel: Liste von Objekten

- gemeinsame Operationen:
  - Hinzufügen, Entfernen, ...
- versch. Implementierungsformen:
  - Feld (array), verkettete Liste, ...



## Deklaration

```
public interface Movable {  
    public void moveBy(int dx, int dy);  
}
```

- Methoden sind automatisch abstrakt
- keine Datenkomponenten

## Klasse implementiert Schnittstellen und ihre abstrakten Methoden

```
public class Point extends Object implements Movable {  
    private int x, y;  
    public void moveBy(int dx, int dy) {  
        x += dx; y += dy;  
    }  
}
```

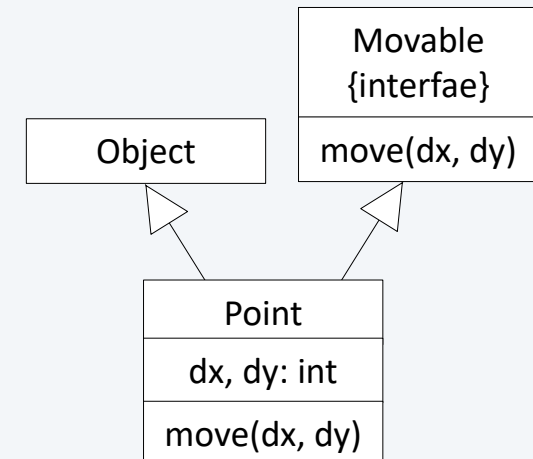
erben von Schnittstellen (beliebig viele)

## Verwendung

```
Movable m = new Point();  
m.moveBy(2, 3);
```

```
Object o = new Point();  
o.moveBy(2, 3);
```

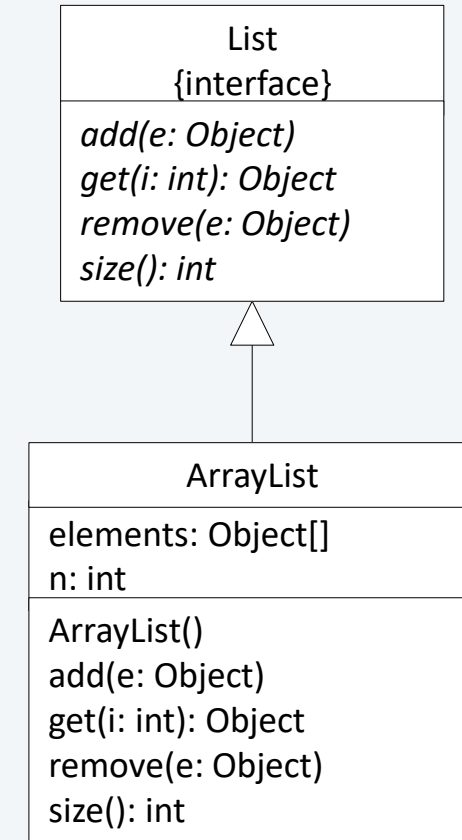
nicht möglich



## Deklarationen der Schnittstelle List und der Klasse ArrayList

```
public interface List {  
    public void add(Object e);  
    public Object get(int i);  
    public void remove(Object e);  
    public int size();  
}
```

```
public class ArrayList implements List {  
    private Object[] elements;  
    private int n;  
  
    public ArrayList() { ... }  
    public void add(Object e) { ... }  
    public Object get(int i) { ... }  
    public void remove(Object e) { ... }  
    public int size() { ... }  
}
```



## Konstruktor und Methoden der Klasse ArrayList

```
ArrayList() {  
    this.elements = new Object[16];  
    this.n = 0;  
}
```

```
public void add(Object e) {  
    if (n == elements.length) {  
        elements = Arrays.copyOf(elements,  
                                   2 * elements.length);  
    }  
    elements[n] = e;  
    n++;  
}
```

```
public Object get(int i) {  
    assert 0 <= i && i < n;  
    return elements[i];  
}
```

```
public int size() {  
    return n;  
}
```

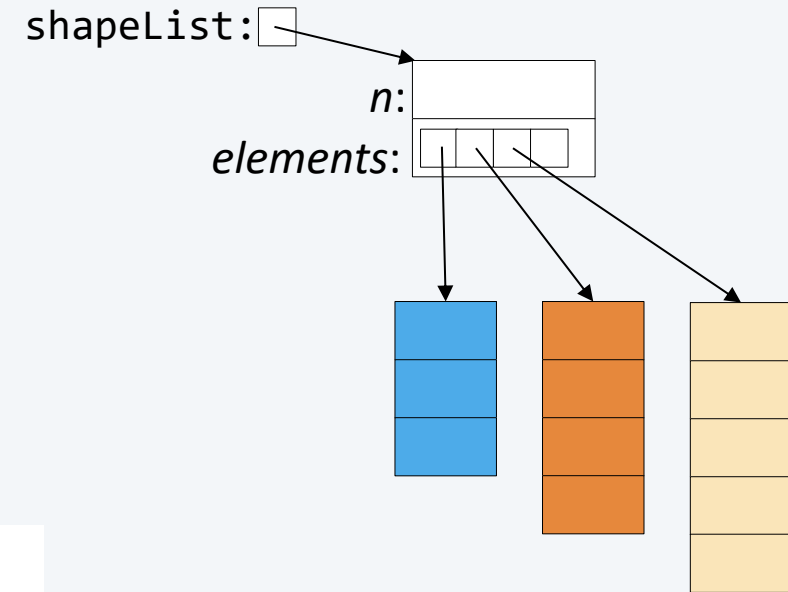
```
public void remove(Object e) {  
    int i = 0;  
    while (i < n && elements[i] != e) {  
        i++;  
    }  
    if (i < n) {  
        for (int j = i; j < n - 1; j++) {  
            elements[j] = elements[j + 1];  
        }  
        n--;  
        elements[n] = null;  
    }  
}
```

## Verwendung eines ArrayList-Objekts

```
List shapeList;
```

```
shapeList = new ArrayList();  
shapeList.add(new Circle());  
shapeList.add(new Rectangle());  
shapeList.add(new ColoredRectangle());  
drawShapes(shapeList);
```

```
void drawShapes(List shapeList) {  
    for (int i = 0; i < shapeList.size(); i++) {  
        Object obj = shapeList.get(i);  
        // draw obj  
    }  
}
```

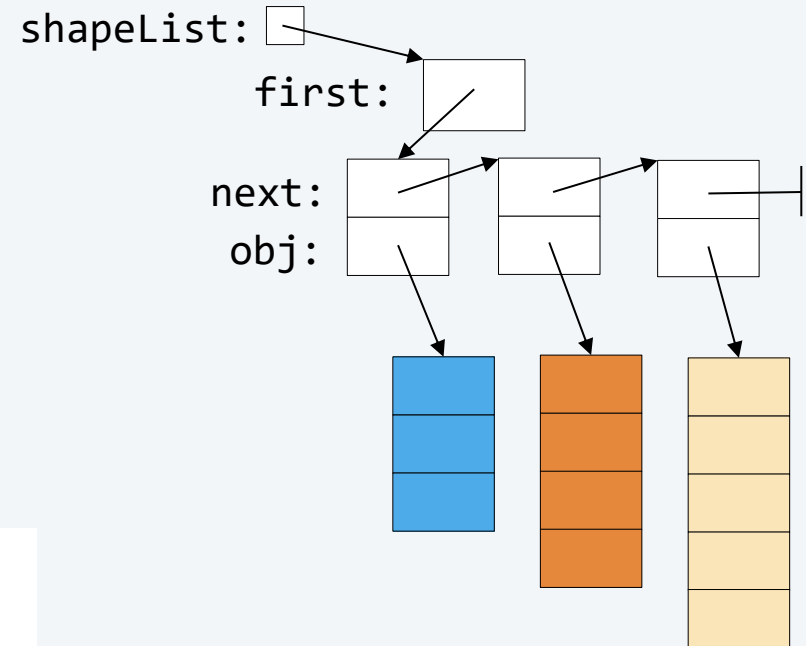


## Verwendung: Austausch des ArrayList-Objekts durch LinkedList-Objekt

```
List shapeList;
```

```
shapeList = new LinkedList();
shapeList.add(new Circle());
shapeList.add(new Rectangle());
shapeList.add(new ColoredRectangle());
drawShapes(shapeList);
```

```
void drawShapes(List shapeList) {
    for (int i = 0; i < shapeList.size(); i++) {
        Object obj = shapeList.get(i);
        // draw obj
    }
}
```



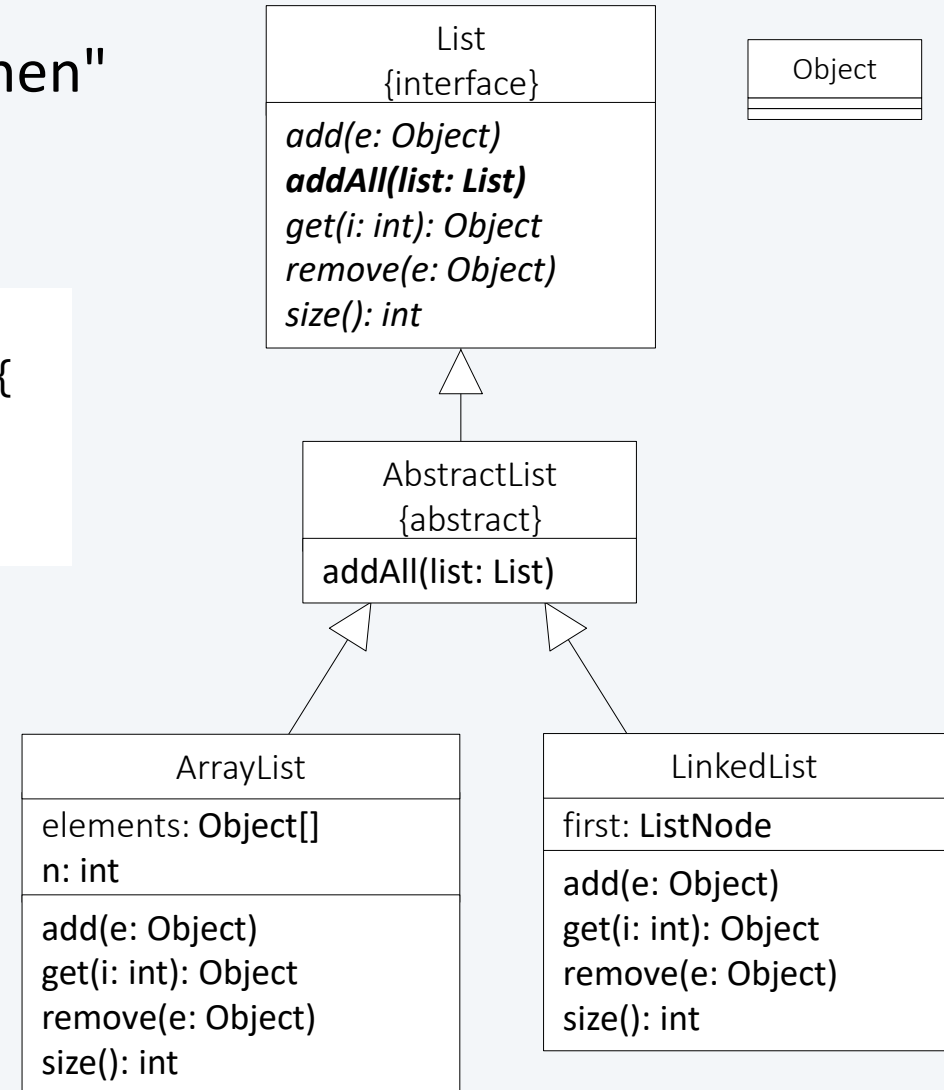
drawShapes() arbeitet mit ArrayList-Objekten und LinkedList-Objekten



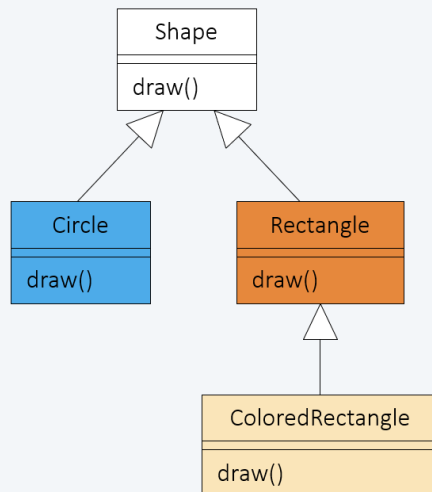
- Faktorisierung gemeinsamer Methoden in abstrakte Klasse "zwischen" Schnittstelle u. konkreten Klassen
- Beispiel: Methode `addAll()`

```
void addAll(List list) {  
    for (int i = 0; i < list.size(); i++) {  
        this.add(list.get(i));  
    }  
}
```

- Typische Hierarchie
  - Schnittstelle
  - abstrakte Klasse
  - konkrete Klassen



## 3 Objektorientierte Programmierung



3.1 Grundlegendes zur OOP

3.2 Vererbung

3.3 Polymorphismus

3.4 Dynamische Bindung

3.5 Abstrakte Klassen

3.6 Typtest und Typumwandlung

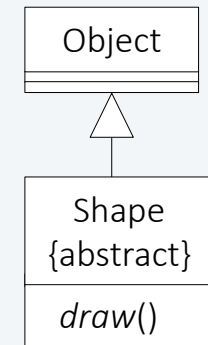
3.7 Vererbungsarten

- Problem bei Verwendung allgemeiner Objektsammlungen

```
Object[] objects;  
int n;
```

```
for (int i = 0; i < n; i++) {  
    objects[i].draw();  
}
```

nicht möglich, wenn in Object  
kein draw() definiert ist



- Lösung durch Typtest und Typumwandlung

```
Object obj;  
Shape s;
```

```
if (obj instanceof Shape) {  
    s = (Shape)obj;  
    s.draw();  
}
```

Typcast

Typumwandlung

- Typcast prüft, ob Objekt hinter einer Variable vom Typ einer bestimmten Klasse oder einer davon abgeleiteten Klasse ist
- Typumwandlung ermöglicht dem Compiler, die Typkompatibilität zu prüfen

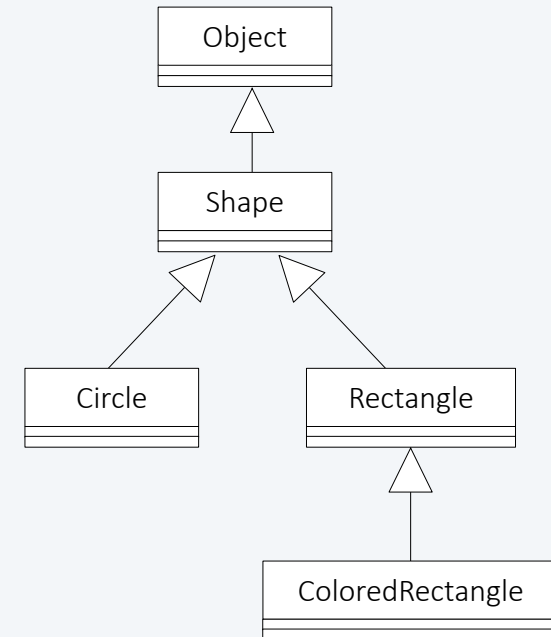
Typstest prüft, ob Objekt hinter einer Variable vom Typ einer bestimmten Klasse oder einer davon abgeleiteten Klasse ist

`obj instanceof ClassType`

liefert true, wenn obj vom dynamischen Typ ClassType (oder einer Unterklasse davon) ist

```
Object obj = new Rectangle();
```

|                                              |       |
|----------------------------------------------|-------|
| <code>obj instanceof Shape</code>            | true  |
| <code>obj instanceof Rectangle</code>        | true  |
| <code>obj instanceof ColoredRectangle</code> | false |
| <code>obj instanceof Circle</code>           | false |



Typcast ermöglicht dem Compiler, die Typkompatibilität zu prüfen

```
Shape s;  
Rectangle r;  
s = new Rectangle();
```

statischer Typ: Shape

```
r = (Rectangle) s ;
```

statischer Typ: Rectangle

Wenn der dynamische Typ nicht Rectangle oder eine von Rectangle abgeleitete Klasse ist, wird zur Laufzeit eine Ausnahmesituation ausgelöst

## Beispiele

```
r = (Rectangle)s;
```

✓

```
c = (Circle)s;
```

✗

ClassCastException: class Rectangle cannot be cast to class Circle

```
cr = (ColoredRectangle)s;
```

✗

ClassCastException: class Rectangle cannot be cast to class ColoredRectangle

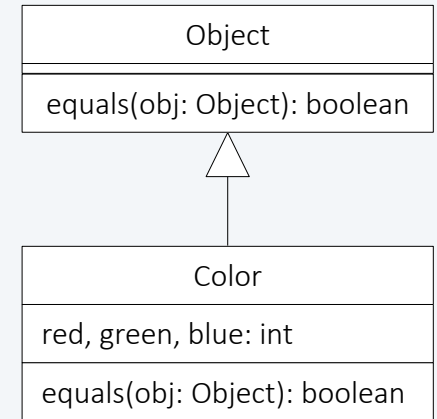
- Method equals in der Wurzelklasse Object

```
public class Object {  
    public boolean equals(Object obj) {  
        return (this == obj);  
    }  
}
```

- abgeleitete Klassen können Methode überschreiben, z.B. um Color-Objekte zu vergleichen

- Verwendung z.B. in einer Methode contains

```
boolean contains(Object obj) {  
    for (int i = 0; i < n; i++) {  
        if (elements[i].equals(obj)) {  
            return true;  
        }  
    }  
    return false;  
}
```



Achtung: Parametertyp darf beim Überschreiben der Methode equals() nicht verändert werden

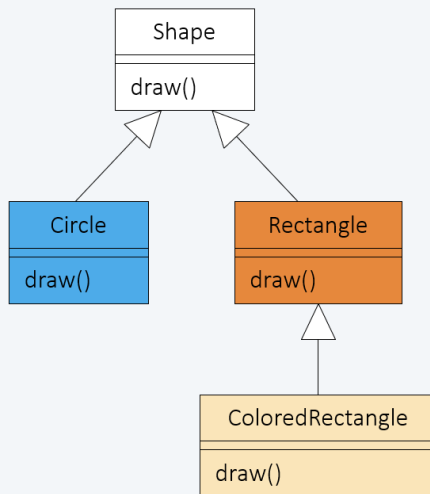
## Überlegungen zur Implementierung der Methode `Color.equals`:

```
Color c1 = new Color(255, 0, 0);  
Color c2 = new Color(255, 0, 0);  
Color c3 = new Color(0, 255, 0);  
Rectangle r1 = new Rectangle(...);
```

|                              |       |
|------------------------------|-------|
| <code>c1.equals(c2)</code>   | true  |
| <code>c1.equals(c3)</code>   | false |
| <code>c1.equals(c1)</code>   | true  |
| <code>c1.equals(r1)</code>   | false |
| <code>c1.equals(null)</code> | false |

```
public class Color extends Object {  
    public boolean equals(Object obj) {  
        if (obj instanceof Color) {  
            Color c = (Color)obj;  
            return red == c.red && green == c.green && blue == c.blue;  
        } else {  
            return false;  
        }  
    }  
}
```

## 3 Objektorientierte Programmierung



3.1 Grundlegendes zur OOP

3.2 Vererbung

3.3 Polymorphismus

3.4 Dynamische Bindung

3.5 Abstrakte Klassen

3.6 Typtest und Typumwandlung

3.7 Vererbungsarten



## Anzahl der Basisklassen

- keine ... unabhängige Klassen
- eine ... einfache Vererbung
- mehrere ... mehrfache Vererbung

## Geschützte Vererbung

- Methoden dürfen überschrieben werden
- einige Methoden sind gegen Überschreiben geschützt

## Prinzip

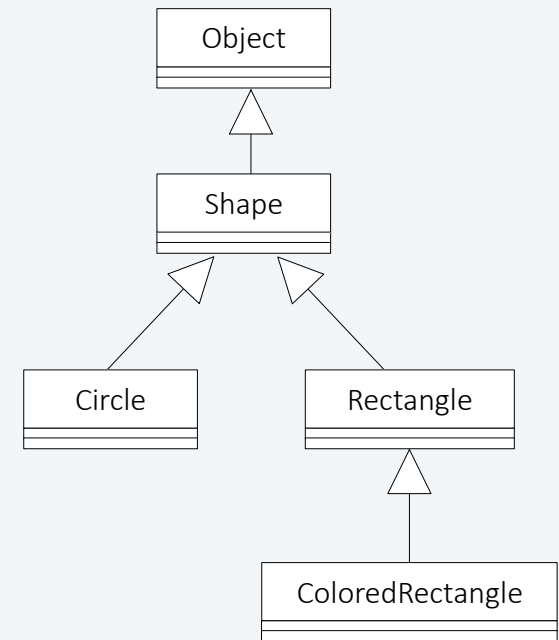
- Wenn von einer Basisklasse B eine Klasse C abgeleitet wird, erbt C alle Eigenschaften und Operationen, die B besitzt

## Anpassungsmöglichkeiten

- neue Datenkomponenten können hinzukommen
- neue Messages/Methoden können hinzukommen
- geerbte Methoden können überschrieben werden

## Konsequenzen

- Klassenhierarchie ist ein Baum
- einfach zu verstehen
- manchmal inadäquate Hierarchie
- Codeverdopplung manchmal unvermeidlich



In der realen Welt finden wir Artefakte, die Eigenschaften mehrerer anderer Artefakte in sich vereinigen

**Beispiel:** ein Schweizermesser ist

- ein Taschenmesser
- ein Flaschenöffner
- ein Korkenzieher
- eine Schere
- ...

Mehrfache Vererbung ist immer dann angebracht, wenn mit Objekten einer Klasse alle Operationen aus zwei oder mehreren Klassen möglich und sinnvoll sind

# Beispiel für mehrfache Vererbung

Ein Window ist ein

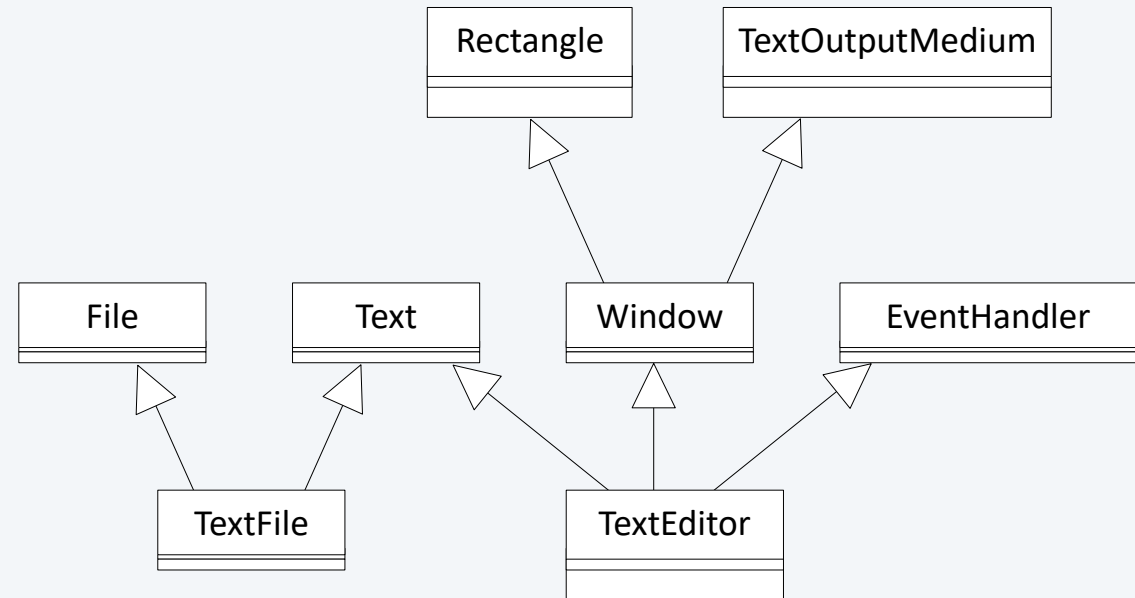
- Rectangle (geometrische Eigenschaften)
- TextOutputMedium (Textausgabe)

Ein TextEditor ist ein

- Window (Anzeigemöglichkeiten)
- Text (Inhalt, Manipulationsmöglichkeiten)
- EventHandler (Behandlung von Benutzereingaben)

Ein TextFile ist ein

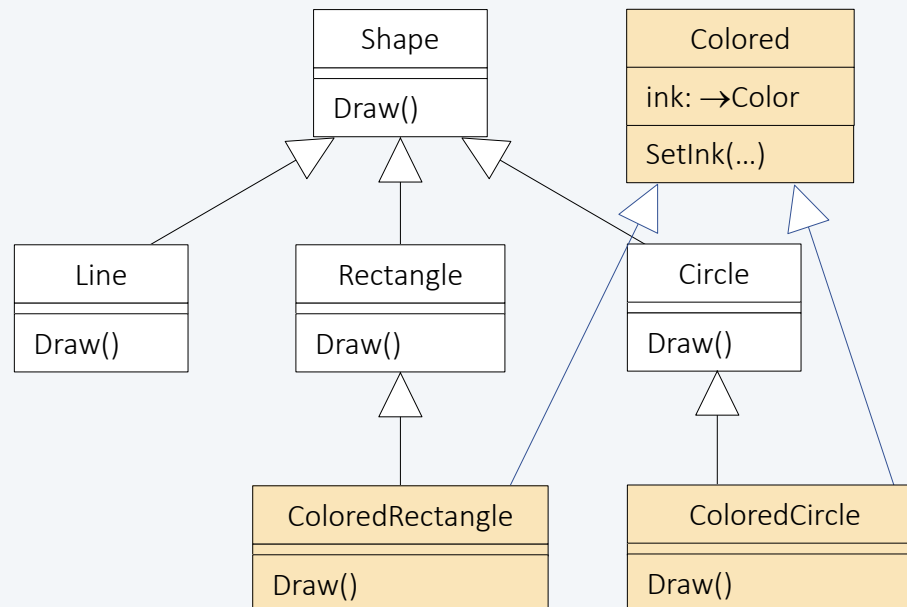
- Text (Inhalt)
- File (Operationen, externe Darstellung)



## Konsequenzen

- Vererbungsbeziehung ist ein gerichteter azyklischer Graph
- Realitätsnahe Modellierung möglich
- Namenskonflikte möglich

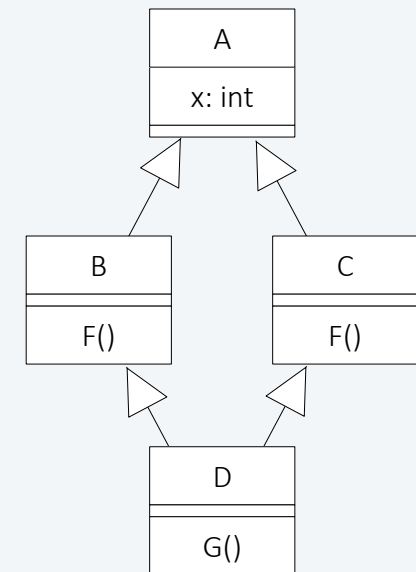
- eine Haupthierarchie (z.B. geometrische Form)
- Eigenschaften werden von kleinen isolierten Klassen geerbt (z.B. Colored)
- diese Form wird als Mix-In bezeichnet



- Klassenbeziehung ist gerichteter azyklischer Graph
- Klassenbeziehungen sind komplexer und nicht so einfach zu verstehen wie Baum bei einfacher Vererbung
- Komplexe Zusammenhänge
- In der Praxis treten viele (sprachspezifische) Probleme auf

## Beispiel: C++

- in Methode `D.G()`: `super→F()` ruft welche Methode?
- für `D`-Objekte: gibt es `x` aus `A` zweimal?
- von `A` kann nur Default-Konstruktor aufgerufen werden



## Klassendefinition:

- Festlegung einer Primär-Basisklasse
- Hinzufügen der Messages der Sekundärklassen

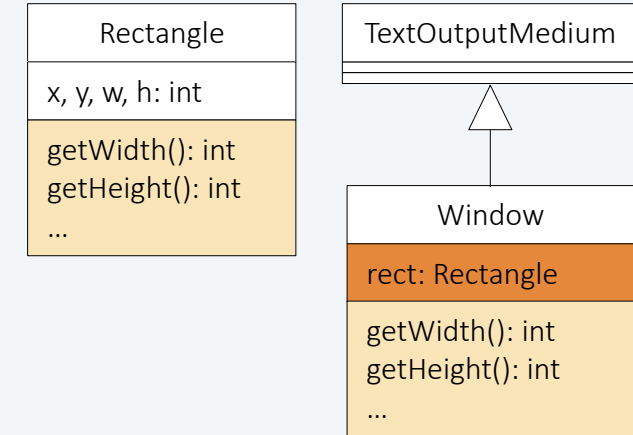
## Klassenimplementierung:

- Erben oder Überschreiben der Methoden der Primär-Basisklasse
- Kopieren der Datenkomponenten und Methoden der Sekundärklassen  
oder:
- Hinzufügen von Objekten der Sekundärklassen und Weiterreichen der Messages

## Beispiel

- Window wird von TextOutputMedium abgeleitet
- Statt einer weiteren Basisklasse Rectangle wird eine Datenkomponente rect eingeführt

```
public class Window extends TextOutputMedium {  
    private Rectangle rect;  
  
    public Window() {  
        this.rect = new Rectangle();  
    }  
  
    public int getWidth() {  
        return rect.getWidth();  
    }  
  
    public int getHeight() {  
        return rect.getHeight();  
    }  
}
```



die Rectangle-Messages werden  
zusätzlich definiert und durch  
Weiterleiten an das Objekt rect  
implementiert (evtl. mit zusätzlichen  
Aktionen)



- manche Messages müssen systemweit gleiche Bedeutung haben
- die entsprechenden Methoden werden an einer Stelle implementiert und dürfen nicht überschrieben werden

## Beispiel

- die Methode getClass ist in der Klasse Object definiert und gibt zur Laufzeit den tatsächlichen Typ (die Klasse) des Objekts zurück
- getClass darf nicht überschrieben werden

```
Shape s = new Rectangle();  
String n = s.getClass().getName();  
IO.println("Class of s is " + n);
```

Class of s is Rectangle

```
public class Object {  
    public final Class getClass() {...}  
}
```

```
public class F extends Object {  
    public Class getClass() {...}  
}
```

Cannot override the final method from Object

- einfache Vererbung bei Klassen (class)
- mehrfache Vererbung bei Schnittstellen (interface)
- da Java-Schnittstellen weder Datenkomponenten noch Methoden<sup>1</sup> enthalten, treten typische Probleme der Mehrfachvererbung nicht auf

<sup>1</sup> seit Java 8 gibt es default-Methoden für Schnittstellen

```
public interface I1 {  
    public void g();  
}
```

```
public interface I2 {  
    public void h();  
}
```

```
public interface I3 {  
    public void g();  
}
```

```
public class C1 {  
    public void f(){...}  
}
```

die Methode `MyClass.g` implementiert die  
abstrakten Methoden `I1.g` und `I3.g`

```
public class MyClass extends C1 implements I1, I2, I3 {  
    public void g() {  
    }  
    public void h() {  
    }  
}
```

```
MyClass o = new MyClass();  
o.f(); // C1.f  
o.g();  
o.h();
```

Objektorientierte Programmierung ist heute eine der wichtigsten Programmiertechniken

## Vorteile

- Vererbung, Klassenbibliotheken und Application Frameworks unterstützen die Wiederverwendung von bereits bestehenden Programmen und Programmkomponenten
- durch Wiederverwendung von Code werden Duplikate vermieden
- Polymorphie und dynamische Bindung helfen, umfangreiche Fallunterscheidungen zu vermeiden
- Abstrakte Klassen fördern die Schaffung von einheitlichen Schnittstellen und die Strukturierung von Softwarearchitekturen
- der Einsatz von Application Frameworks führt zu Systemen mit ähnlichen Benutzerschnittstellen und fördert so die Einheitlichkeit des Erscheinungsbildes von Programmen (Standardisierung)

## Nachteile

- Neben der Beherrschung einer objektorientierten Sprache, sind Kenntnisse des Aufbaus von Klassenbibliotheken u/o Application Frameworks erforderlich
- Dynamisch erzeugte Objekte erhöhen den Speicherbedarf
- Dynamische Bindung kann die Programmlaufzeit verlängern